

I. FROM UNDERSTANDING TO CREATION

If BERT is a "Reader" that analyzes existing text, **GPT** is a "Writer" that predicts the future. Developed by OpenAI, the GPT series (from GPT-1 to GPT-4) proved that if you give a model enough data and enough parameters, it develops "Emergent Abilities"—the ability to reason, solve math problems, and write software without being explicitly programmed to do so.

The Fundamental Goal: Given a sequence of words, predict the single most likely **next token**.

II. ARCHITECTURE: THE DECODER-ONLY DESIGN

Unlike the original Transformer which had an Encoder and a Decoder, GPT uses only the **Decoder** blocks.

2.1. Causal Masking (The "Blindfold")

The key difference between GPT and BERT is **Masking**.

- **BERT (Bidirectional):** Sees the whole sentence at once.
- **GPT (Unidirectional/Causal):** When predicting the 5th word, it is mathematically forbidden from seeing the 6th, 7th, or 8th words. It only looks "backwards" at the history it has already generated.

2.2. Scaling Laws

GPT models are defined by their **Parameters** (the "synapses" of the AI).

- **GPT-2:** 1.5 Billion parameters.
- **GPT-3:** 175 Billion parameters. As the model grows, it stops just "memorizing" and starts "interpolating" logic, allowing it to handle tasks it has never seen before (Zero-Shot Learning).

III. THE TRAINING PHASES: PRE-TRAINING VS. ALIGNMENT

A modern GPT model (like the one powering your RAG) goes through two distinct stages of education:

3.1. Generative Pre-training (The "Library" Phase)

The model reads a massive portion of the internet (Common Crawl, Wikipedia, GitHub). It learns grammar, facts, and coding patterns by playing a trillion rounds of "Predict the next word."

- *Result:* A "Base Model" that is very smart but hard to talk to (it might just continue your sentence rather than answering your question).

3.2. Instruction Fine-Tuning & RLHF (The "School" Phase)

The model is trained on pairs of [Question] and [Correct Answer].

- **RLHF (Reinforcement Learning from Human Feedback):** Humans rank multiple AI answers from best to worst. This "aligns" the AI to be helpful, honest, and harmless.

IV. KEY CONCEPTS: TEMPERATURE AND TOP-P

When a GPT model generates text, it doesn't just pick the #1 most likely word. It uses "Sampling" to stay creative.

- **Temperature:** Controls randomness.
 - *Low (0.1):* Very predictable and factual.
 - *High (0.8+):* Creative, diverse, but prone to "hallucinations."
- **Tokens:** GPT doesn't see "words"; it sees "tokens" (chunks of characters). "University" might be one token, while a rare word might be split into three.

V. PRACTICAL LAB: GENERATING TEXT WITH HUGGING FACE

This code shows how to load a GPT-style model (like **Llama-3** or **Mistral**) to generate a technical summary for one of your 50 files.

Python

```
from transformers import AutoModelForCausalLM, AutoTokenizer
```

```
import torch
```

```
# 1. Load a pre-trained Decoder model (e.g., Mistral-7B)
```

```
model_name = "mistralai/Mistral-7B-v0.1"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForCausalLM.from_pretrained(model_name, torch_dtype=torch.float16,  
device_map="auto")
```

```
# 2. Define a prompt for your RAG project
```

```
prompt = "Summarize the importance of Data Engineering in 3 bullet points:"
```

```
# 3. Tokenize and Generate
```

```
inputs = tokenizer(prompt, return_tensors="pt").to("cuda")
```

```
output = model.generate(**inputs, max_new_tokens=100, temperature=0.7)
```

```
# 4. Decode the result back to human text
```

```
print(tokenizer.decode(output[0], skip_special_objects=True))
```

VI. GPT IN THE RAG PIPELINE

In your 50-file project, GPT acts as the "**Synthesizer**":

1. **Input:** It receives the User Query + the Relevant Chunks found by BERT (File 32).
2. **Logic:** It reads the chunks and extracts the answer.
3. **Output:** It reformats the technical data into a natural, conversational response for the student.